WiggleBoi

Operating System Concepts in a Zero-Dependency

Terminal Game Engine with 3D Raycasting

Ritesh Patil
230056
BTech CSAI
Rishihood University

Meghavi Rao
230044
BTech CSAI
Rishihood University

Samarth Shekhar
230042
BTech CSAI
Rishihood University

Repository: https://github.com/Champion1102/wiggleboi

Abstract — This paper presents WiggleBoi, a terminal-based snake game engine implemented in approximately 3,400 lines of pure C with zero standard library dependencies. Every component—from basic arithmetic operators to a textured 3D raycasting renderer—is built from scratch using only raw operating system calls. The project serves as a practical demonstration of core OS concepts: memory management via `mmap`, process management via `fork`/`exec`, I/O management through raw terminal control and double-buffered rendering, file system interaction via direct `open`/`read`/`write` syscalls, and robust error handling through signal-driven graceful shutdown. The system features bitwise-only arithmetic (addition, multiplication, division implemented without +, *, / operators), procedural texture generation, AI opponents with BFS pathfinding, and a first-person 3D raycasting mode with textured walls, floors, ceilings, and pixel-art sprites—all within ~90 KB binary size and ~245 KB runtime memory.

Keywords — *operating systems, memory management, process management, I/O management, file systems, systems programming, raycasting, terminal rendering*

I. Introduction

Modern software development abstracts away the operating system through layers of standard libraries and frameworks. Developers call `printf` without considering how bytes reach the terminal, use `malloc` without understanding virtual memory pages, and rely on `math.h` without knowing how arithmetic is performed at the hardware level.

WiggleBoi was designed as a comprehensive exercise in building a complex interactive application from first principles, using *only* raw OS system calls. Every standard library function is replaced with a hand-rolled implementation, exposing and demonstrating the OS concepts that underpin modern software.

Design Constraints

- No standard library: no `printf`, `malloc`, `strlen`, `math.h`
- Only raw syscalls: `mmap`, `write`, `read`, `fork`, `select`, `ioctl`
- Arithmetic from bitwise operators only: no `+`, `-`, `*`, `/`, `%`
- Cross-platform: macOS and Linux

II. Methodology

Development followed an incremental, bottom-up approach where each layer was built only from primitives already implemented.

II-A Development Phases

1) **Foundation** — Custom memory allocator (`mmap`), string utilities (`str_len`, `int_to_str`), and bitwise arithmetic engine (`my_add`, `my_mul`, `my_div`, `my_mod`).
2) **Terminal I/O** — Raw keyboard input via `tcsetattr`, non-blocking reads via `select`, and a double-buffered truecolor renderer with half-block pixel resolution doubling.
3) **Game Engine** — Entity pool with stack-based free list, spatial grid for $O(1)$ collision detection, snake as a doubly-linked list, state machine game loop.
4) **Gameplay** — Four game modes, seven item types, power-up timers, obstacle patterns, score persistence via binary file I/O.
5) **AI System** — Four AI snakes with depth-limited BFS pathfinding, three personality types, distributed scheduling to spread CPU load.
6) **Audio** — Cross-platform sound via `fork`/`exec` process isolation, looping background music via daemon processes.
7) **3D Raycasting** — Fixed-point DDA raycaster with Bhaskara trigonometry, procedural texture generation, per-pixel texture mapping, multi-factor lighting, sprite projection.
8) **Robustness** — Signal handlers for graceful shutdown, bounds checking, division-by-zero guards, pool exhaustion safety.

II-B Design Philosophy

Each module was designed to be self-contained with minimal coupling. Shared state is limited to the spatial grid (queried by the game loop, AI, and raycaster) and the entity pool. All modules communicate through function calls rather than shared global mutation.

III. System Architecture

The application comprises 13 modular source files plus the main game loop. Table I summarizes the structure.

TABLE I: Module Structure and Responsibilities

Module	Responsibility	Lines
main.c	State machine, game loop, rendering	1,302
ray.c	DDA raycaster, textured 3D rendering	555
ai.c	AI snakes, BFS pathfinding	457
screen.c	Double-buffered terminal renderer	315
texture.c	Procedural texture generation	290
math.c	Bitwise arithmetic, RNG	143
fx.c	Particle system, screen shake	125
save.c	Binary file I/O for high scores	98
sound.c	Audio via fork/exec	87
entity.c	Entity pool, spatial grid	85
keyboard.c	Raw-mode input, escape parsing	81
string.c	String length, int-to-string	49
memory.c	`mmap`/`munmap` allocator	32
	Total	**~3,400**

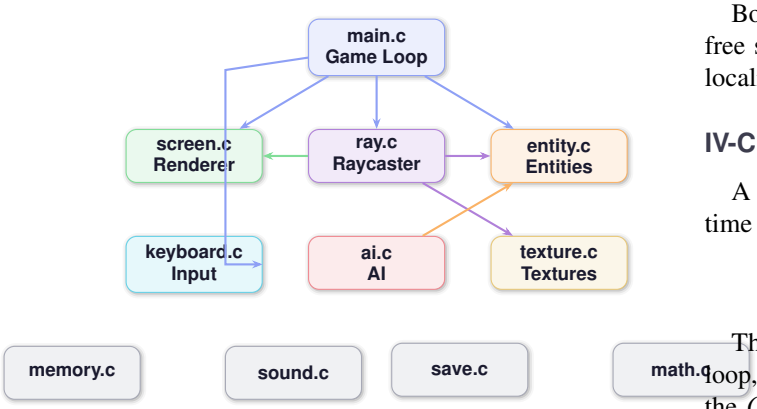

Fig. 1: Module dependency graph. Arrows indicate primary data flow between subsystems.

IV. Memory Management

IV-A Custom mmap Allocator

The project replaces `malloc`/`free` with direct `mmap`/`munmap` system calls, requesting anonymous memory pages directly from the kernel:

mmap-Based Allocator (memory.c)

```c
void *mem_alloc(unsigned long size) {
    void *p = mmap(0, size,
        PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANON, -1, 0);
    if (p == MAP_FAILED) return 0;
    return p;
}
void mem_free(void *ptr, unsigned long size) {
    munmap(ptr, size);
}
```

`MAP_ANON` requests anonymous memory not backed by any file. `MAP_PRIVATE` ensures writes are process-private. The kernel returns zeroed, page-aligned memory (4 KB granularity). Failure is detected via `MAP_FAILED` check.

IV-B Entity Pool — Zero-Fragmentation Allocator

For frequent small allocations (snake segments, food, obstacles), a fixed-size pool of 1,024 pre-allocated entities avoids fragmentation entirely:

Stack-Based Free List (entity.c)

```c
static struct Entity pool[1024];
static unsigned char free_stack[1024];
static int free_top;

int ent_alloc(void) {
    if (free_top <= 0) return -1;
    free_top--;
    int id = free_stack[free_top];
    pool[id].flags = ENT_ALIVE;
    return id;
}
void ent_free(int id) {
    free_stack[free_top] = id;
    free_top++;
}
```

Both allocation and deallocation are $O(1)$—push/pop on the free stack. The pool is contiguous in memory, providing cache locality.

IV-C Spatial Grid — O(1) Collision Detection

A 2D array indexed by grid coordinates provides constant-time collision lookup:

$$\texttt{grid}[y \times w + x] = \texttt{entity_id} \tag{1}$$

This grid is allocated via `mmap` and shared by the game loop, AI system, and raycaster for wall detection—eliminating the $O(n)$ cost of scanning all entities.

IV-D Memory Budget

TABLE II: Runtime Memory Allocation

Component	Size
Screen buffers (front + back + write)	85 KB
Texture data (8 wall + 4 sprite textures)	102 KB
Pixel buffer (3D rendering)	25 KB
Entity pool + free stack + spatial grid	12 KB
Trig tables + depth buffers	16 KB
AI state, particles, save data	5 KB
Total	**~245 KB**

V. Process Management

V-A Audio via fork/exec

Sound playback uses Unix process creation to achieve asynchronous, fire-and-forget audio without linking to audio libraries:

Process-Isolated Audio (sound.c)

```c
void snd_play(const char *path) {
    signal(SIGCHLD, SIG_IGN);  /* auto-reap */
    if (fork() == 0) {
        setsid();              /* new session  */
        close(0);              /* detach stdin  */
        int fd = open("/dev/null", O_WRONLY);
        dup2(fd, 1);           /* silence stdout */
        dup2(fd, 2);           /* silence stderr */
        char *args[] = {player, path, 0};
        execvp(player, args);  /* replace child */
        _exit(1);
    }
}
```

Key process management concepts demonstrated:

- **fork()** — Creates child process with copy-on-write memory
- **execvp()** — Replaces child's address space with the audio player binary
- **setsid()** — Creates a new session, detaching from the parent's terminal
- **SIGCHLD/SIG_IGN** — Prevents zombie processes by auto-reaping children

V-B Background Music Daemon

Looping music uses a two-level process hierarchy: an outer daemon process that repeatedly forks a player child, waits for completion, then loops:

Process Hierarchy

`Game (parent)` → `fork()` → `Music Daemon` → `fork()` → `afplay/paplay`

The daemon's PID is stored. On shutdown: `kill(-pid, SIGTERM)` sends the signal to the *entire process group*, terminating both the daemon and any active player child in one call.

V-C Signal Handling and Graceful Shutdown

The game registers handlers for four termination signals:

Signal-Safe Shutdown (main.c)

```c
static void on_signal(int sig) {
    (void)sig;
    snd_music_stop();    /* kill audio procs */
    scr_shutdown();      /* restore terminal */
    kb_restore();        /* restore tty mode */
    _exit(0);
}
/* In main(): */
signal(SIGINT,  on_signal);
signal(SIGTERM, on_signal);
signal(SIGHUP,  on_signal);
signal(SIGQUIT, on_signal);
```

Without these handlers, Ctrl+C would leave the terminal in raw mode with a hidden cursor—requiring manual `reset` to recover.

VI. I/O Management

VI-A Raw Terminal Input

Standard terminal input is line-buffered (canonical mode)—characters are not available until Enter is pressed. The game disables this for real-time key response:

Raw Mode Configuration (keyboard.c)

```c
void kb_raw(void) {
    tcgetattr(0, &orig);        /* save state */
    struct termios raw = orig;
    raw.c_lflag &= ~(ICANON   /* no buffering  */
                     | ECHO     /* no echo       */
                     | ISIG);   /* no Ctrl+C     */
    raw.c_cc[VMIN]  = 0;       /* non-blocking  */
    raw.c_cc[VTIME] = 0;
    tcsetattr(0, TCSAFLUSH, &raw);
}
```

VI-B Non-Blocking Reads via select()

The `select()` system call checks if stdin has data available without blocking:

Non-Blocking Key Read (keyboard.c)

```c
int kb_read(void) {
    fd_set fds;
    struct timeval tv = {0, 0}; /* zero timeout */
    FD_ZERO(&fds);
    FD_SET(0, &fds);
    if (select(1, &fds, 0, 0, &tv) <= 0)
        return 0;  /* no input available */
    /* ... read and parse key ... */
}
```

Arrow keys produce multi-byte escape sequences (`ESC [ A/B/C/D`). The parser uses a 5 ms `select()` timeout to distinguish standalone ESC from arrow key sequences.

VI-C Double-Buffered Terminal Output

The renderer maintains two screen buffers and performs differential output:

1) Game logic writes to the *back buffer*
2) `scr_present()` diffs back vs. front buffer
3) Only changed cells generate ANSI escape sequences
4) All output is accumulated in a 65 KB staging buffer
5) A *single* `write(1, buf, len)` syscall flushes the entire frame

Synchronized Output mode (`ESC[?2026h/l`) wraps the frame update, preventing the terminal from rendering partial frames (tearing).

VI-D Half-Block Pixel Rendering

The Unicode upper half-block character (`U+2580`) enables each terminal cell to display two vertical pixels by encoding the top pixel as foreground color and the bottom pixel as background color. This **doubles vertical resolution** from terminal rows to usable pixels.

VII. File System Operations

VII-A High Score Persistence

Score data is persisted using direct file system calls without `stdio`:

Binary File I/O (save.c)

```c
/* Load scores */
int fd = open(path, O_RDONLY);
if (fd >= 0) {
    int n = read(fd, entries,
                 sizeof(entries));
    num_entries = n / sizeof(ScoreEntry);
    close(fd);
}

/* Save scores */
int fd = open(path,
    O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, entries,
      num_entries * sizeof(ScoreEntry));
close(fd);
```

VII-B Path Construction

The save file path is constructed by reading the `HOME` environment variable and appending `/.wiggleboi_scores`. If `HOME` is unavailable, the system falls back to `/tmp`—ensuring functionality across different environments.

VII-C Sorted Insertion

New scores are inserted in descending order using in-place array shifting. The file is truncated (`O_TRUNC`) and rewritten on each save, maintaining a maximum of 10 entries at 8 bytes each (80 bytes total).

TABLE III: File System Operations Used

Syscall	Purpose
`open(path, flags, mode)`	Open/create score file
`read(fd, buf, n)`	Load binary score data
`write(fd, buf, n)`	Persist score data
`close(fd)`	Release file descriptor
`O_CREAT | O_TRUNC`	Create if missing, truncate on wr

VIII. Security and Error Handling

VIII-A Signal-Driven Graceful Shutdown

Four signals are handled to ensure the terminal is always restored to a usable state, even on unexpected termination (Section V-C). Without this, a crash would leave the terminal in raw mode with invisible cursor—effectively "bricking" the user's shell session.

Additionally, `atexit(snd_music_stop)` ensures the music daemon is killed even on normal exit paths.

VIII-B Arithmetic Safety

- **Division by zero:** `fp_div()` returns `0x7FFFFFFF` (max int) when divisor is zero, preventing crashes in the raycaster
- **Perpendicular distance:** Clamped to minimum 1 after computation (`if (perp <= 0) perp = 1`)
- **Cosine correction:** Guarded against zero (`if (cos_diff <= 0) cos_diff = 1`)
- **Floor casting:** Pixel distance from horizon guarded (`if (dy_pix <= 0) dy_pix = 1`)

VIII-C Bounds Checking

- **Grid boundaries:** `grid_occupied()` returns 1 (occupied) for out-of-bounds coordinates, acting as implicit walls
- **Entity pool exhaustion:** `ent_alloc()` returns -1 when the pool is full; callers check before use
- **Screen bounds:** All pixel writes check `x < pix_w` and `y < pix_h` before buffer access
- **Depth buffer:** Ray column indices clamped to 512 maximum
- **mmap failure:** `mem_alloc()` checks for `MAP_FAILED` and returns null

VIII-D Process Isolation for Audio

Audio playback runs in isolated child processes via `fork()/setsid()/execvp()`. If the audio player crashes, the game is completely unaffected. The child's stdin/stdout/stderr are redirected to `/dev/null`, preventing any interference with terminal rendering.

IX. Bitwise Arithmetic Engine

All five arithmetic operations are implemented from scratch using only bitwise operators (`&`, `|`, `^`, `~`, «, »), mirroring hardware ALU logic:

IX-A Addition — Ripple Carry Adder

XOR computes the sum without carry; AND identifies carry positions; left-shift propagates carries:

Bitwise Addition (math.c)

```c
int my_add(int a, int b) {
    int carry;
    while (b) {
        carry = a & b;
        a = a ^ b;
        b = carry << 1;
    }
    return a;
}
```

IX-B Multiplication — Shift-and-Add

For each set bit in the multiplier, the multiplicand (shifted to the corresponding position) is added to the accumulator:

$$a \times b = \sum_{i=0}^{31} \big(b_i \cdot (a \ll i)\big) \quad \text{where } b_i = (b \gg i) \ \& \ 1 \tag{2}$$

IX-C Division — Binary Long Division

The restoring algorithm processes dividend bits from MSB to LSB, building the quotient one bit at a time. Modulo is derived as $a \bmod b = a - (a/b) \times b$, composing division, multiplication, and subtraction.

IX-D Application

The LCG random number generator uses these bitwise primitives for its core computation:

$$\text{seed} = \texttt{u_add}\big(\texttt{u_mul}(\text{seed},\ 1103515245),\ 12345\big) \tag{3}$$

X. 3D Raycasting Engine

X-A Fixed-Point Arithmetic

All 3D calculations use 10-bit fixed-point integers (no floating-point hardware required):

$$\text{FP_ONE} = 2^{10} = 1024 \qquad v_{\text{real}} \approx \frac{v_{\text{fp}}}{1024} \tag{4}$$

Multiplication uses 64-bit intermediates to prevent overflow:

```
return (int)(((long long)a * b) >> 10);
```

X-B Trigonometric Tables

Sine and cosine are precomputed using Bhaskara I's 7th-century rational approximation [2], achieving $< 0.2\%$ error:

$$\boxed{\sin(\theta) \approx \frac{16\,\theta\,(\pi - \theta)}{5\pi^2 - 4\,\theta\,(\pi - \theta)}} \tag{5}$$

The angle space uses 1,024 discrete steps (~0.35°/step). Cosine is derived via 256-step phase offset.

X-C DDA Raycasting

The Digital Differential Analyzer [1] casts one ray per screen column, stepping through grid cells until hitting a wall:

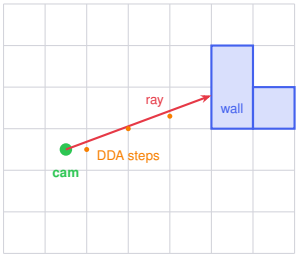

Fig. 2: DDA traversal: the ray steps from grid line to grid line, checking wall occupancy via the spatial grid in $O(1)$ per step.

Fisheye correction uses: $d_{\perp} = d_{\text{raw}} \times \cos(\alpha_{\text{ray}} - \alpha_{\text{camera}})$

X-D Procedural Texture System

Eight 64×64 wall/floor/ceiling textures and four 16×16 sprite textures are generated algorithmically at startup using a deterministic hash function for noise:

Hash-Based Texture Noise (texture.c)

```
int tex_hash(int x, int y, int seed) {
    int h = x * 374761393
          + y * 668265263 + seed;
    h = (h ^ (h >> 13)) * 1274126177;
    h = h ^ (h >> 16);
    return h;
}
```

Texture types include brick (staggered grid + mortar), stone (blocks + seams), metal (panels + rivets), snake scales (diamond lattice), and diagonal warning stripes. Sprites include an apple, golden star, purple skull, and lightning bolt—all generated from mathematical formulas.

X-E Multi-Factor Lighting

Wall brightness combines four factors:

$$L = \frac{L_{\text{dist}} \times L_{\text{dir}} \times L_{\text{vert}} \times L_{\text{flicker}}}{256^3} \tag{6}$$

where L_{dist} is quadratic distance attenuation, L_{dir} is directional shading (N/S vs E/W), L_{vert} is a vertical gradient, and L_{flicker} adds subtle torch-like animation to obstacle walls. Colored fog blends pixel colors toward dark blue $(8, 10, 20)$ at distance.

X-F Rendering Pipeline

Fig. 3: 3D rendering pipeline. The entire frame is computed in-memory and flushed via a single `write()` syscall.

XI. AI System

Up to four AI snakes spawn as score increases (at 8, 16, 25, 35 points). Each has a personality:

- **Aggressive** — BFS toward food
- **Cautious** — flees if player is within Manhattan distance 6
- **Random** — random wandering with occasional BFS

Pathfinding uses depth-limited BFS (max depth 5, queue size 128). Computation is distributed: only one AI runs expensive pathfinding per tick via `(idx & 3) == (tick & 3)`, spreading CPU cost across frames.

XII. Future Scope

1) **Networked Multiplayer** — Implement a simple TCP-based protocol using `socket()`/`connect()`/`accept()` syscalls for head-to-head play, extending the zero-dependency constraint to networking.
2) **Procedural Map Generation** — Replace the static rectangular arena with algorithmically generated maze layouts, adding corridors, rooms, and doorways to the 3D raycasting mode.
3) **WebAssembly Port** — Compile to WASM with a browser-based terminal emulator frontend, making the game playable in any web browser without installation.
4) **GPU-Accelerated Terminal Rendering** — Leverage the Kitty graphics protocol to transmit rendered pixel data directly as images, bypassing ANSI escape sequence overhead for higher frame rates.
5) **Level Editor** — A terminal-based visual editor for designing custom arena layouts, stored as compact binary map files.
6) **Extended Texture System** — Animated wall textures (flowing water, flickering torches), mipmapping for distant texture quality, and a larger texture atlas.
7) **Sound Synthesis** — Replace external audio files with raw PCM waveform generation written directly to `/dev/audio` or via ALSA syscalls, achieving fully self-contained audio.

XIII. Performance Metrics

TABLE IV: Technical Metrics

Metric	Value
Source lines	~3,400
Binary size	~90 KB
Runtime memory	~245 KB
Source files	26 (.c + .h)
Standard library calls	0
Entity pool capacity	1,024 slots
Trig table entries	1,024
Texture resolution	64×64 (walls), 16×16 (sprites)
Fixed-point precision	10-bit fractional
View distance	40 cells
Color depth	24-bit truecolor (16.7M)

XIV. Conclusion

WiggleBoi demonstrates that complex interactive graphical applications can be constructed from minimal OS-level primitives. By replacing every standard library function with hand-rolled implementations—from addition via ripple-carry adders to 3D texture mapping via fixed-point raycasting—the project provides a practical exploration of operating system concepts.

The five core OS areas—memory management (mmap allocator, entity pool, spatial grid), process management (fork/exec audio, signal handlers, process groups), I/O management (raw terminal control, double-buffered rendering, non-blocking input), file systems (binary score persistence via direct syscalls), and security/error handling (graceful shutdown, bounds checking, arithmetic guards)—are not studied in isolation but integrated into a functioning, playable application.

The result is a self-contained game engine that compiles to under 100 KB, runs with approximately 245 KB of memory, and delivers a textured first-person 3D experience inside a terminal emulator—all without a single standard library call.

Source: https://github.com/Champion1102/wiggleboi

References

[1] F. Permadi, "Ray-Casting Tutorial for Game Development and Other Purposes," 1996. [Online]. Available: https://permadi.com/1996/05/ray-casting-tutorial-table-of-contents/

[2] R. C. Gupta, "On the school of Aryabhata: Bhaskara I's approximation to sine," *Ganita*, vol. 18, no. 2, pp. 13–18, 1967.

[3] IEEE/Open Group, "mmap — map pages of memory," in *The Open Group Base Specifications*, Issue 7, 2018. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/functions/mmap.html

[4] ECMA International, "Control Functions for Coded Character Sets," ECMA-48, 5th ed., 1991. [Online]. Available: https://ecma-international.org/publications-and-standards/standards/ecma-48/